SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Name	Chennuri Sumedha
Register Number	1971260029
Course Code & Name	C Programming (CSA0210)
Slot	C
Assignment / Task Title	Q1 Comprehensive System Design and Implementation Challenge — District Blood Bank Network
GitHub Repository URL	https://github.com/sumedha-2805/CSA0210.git
Submission Date	13-11-2026

Instructions to Students

1. Complete every sectionfor eachassigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

Case study: A district has one central blood bank and several hospital storage units. Donor records hold ID, name, blood group, age, phone and date of last donation; stock records hold a blood group and units available. The program must register donors, find eligible donors of a group, update stock after every donation and issue, and keep all data in files. **Assumption used throughout:** a donor is eligible if age is 18 to 65 and at least 90 days have passed since the last donation. **Task map:** Task 1 = A1, A3 | Task 2 = A2, B1 | Task 3 = A4, B4 | Task 4 = A5, B3 | Task 5 = B2.

TASK 1 — Functions, Interfaces and Pointers (Parts A1, A3)

Task / Problem Statement

Break the blood bank program into functions, draw which function calls which, design the parameter passing (value, pointer, const pointer), return more than one value in two ways, and explain pointer expressions on the stock array.

Aim

To design a modular blood bank program with justified interfaces, and to show pointer operations and pointer-returning functions with a working program.

Concept / Theory

A function is a named block that does one job; breaking a program into functions makes it easier to test and reuse. C passes arguments by value (a copy). To let a function change the caller's data, or avoid copying a big structure, the address is passed as a pointer; const promises the function will not modify the data.

A pointer holds an address. For an array, p + k moves k elements, and p[k] is the same as *(p + k). A function can return only one value, so extra results are given through pointer parameters or inside a struct.

Procedure / Algorithm

A1.1 — Functions and structure chart

```
main
 +-- load_all()
 +-- run_command()
```

```
                          reads donors.txt (text) and stock.dat (binary)
                          checks argc/argv, chooses the action
  |     +--  register_donor()  --> add_donor() --
  |     +-- find_donors() +--  > is_eligible() --
  |     issue_blood()     +--  > update_stock() -
  |     donate_blood()           -> update_stock()
  +--  save_all()           --> save_donors(), save_stock()
+-- free_all()              releases the donor list
```

A1.2 — Prototypes and parameter decisions

Prototype	Pass by	Justification
int add_donor(DonorList *list, int id, const char *group)	list: pointer; id: value; group: const pointer	The list is changed, so its address is needed. id is one int, a copy is cheap. group is only read, so const protects it.
int is_eligible(const Donor *d)	const pointer	Avoids copying the whole struct; const because the check never changes the donor.
int find_donors(const DonorList *l, const char *group, int ids[], int max)	l, group: const pointer; ids: pointer	l and group are read only. ids is an output array, so it must not be const.
int update_stock(int stock[], int g, int units)	array = pointer; g, units by value	The array element is modified, so no const. g and units are small copies.
int save_donors(const DonorList *l, const char *file)	const pointers	Only reads the list and the file name.
int load_donors(DonorList *l, const char *file)	pointer, const pointer	Fills the list (modified); file name only read.
int *find_lowest(int stock[], int n)	pointer, value	Returns the address of the element so the caller can update it.

A1.3 — Returning more than one value (count of eligible donors and ID of the youngest eligible donor)

	Design 1: pointer parameter	Design 2: return a structure
Prototype	int eligible_ptr(const Donor d[], int n, int *youngest_id)	EligResult eligible_struct(const Donor d[], int n)
How	returns count, writes ID through the pointer	returns {count, youngest_id} together
Good	No extra type; fine for large results	Self-explanatory call; no chance of a bad pointer
Bad	Caller must pass a valid address; output hidden in the parameter list	Struct is copied (only 8 bytes here, so cheap)
Time / space	O(n) / O(1)	O(n) / O(1)

Alternative — global variables. The functions would read a global donor array and count and need no parameters. Running time and memory stay the same (O(n), O(1) extra), but coupling becomes tight: every function depends on hidden shared data, testing needs the globals set up in the right order, and the functions cannot be reused for a second list (for example another hospital).

A3.1 — Pointer expressions for int stock[8]; int *p = stock;

- p + 3 is the address of stock[3], 3 x sizeof(int) = 12 bytes after stock[0] (pointer arithmetic moves in whole elements).
- *(p + 3) is the value stored at that address, i.e. stock[3].
- p[3] is another way of writing *(p + 3); both give stock[3].
- *p++ gives the value at *p (stock[0]) and then moves p on to stock[1].

```
index :     0     1     2     3     4     5     6     7
stock : [ 40 ][ 25 ][ 12 ][ 8 ][ 30 ][ 17 ][ 5 ][ 22 ]
addr :    1000 1004 1008 1012 1016 1020 1024 1028            (example addresses, 4 bytes apart)
          ^p                ^p+3
After *p++ : p -> 1004 (stock[1])
```

A3.2 — Pointer-returning function. find_lowest() returns &stock[i] of the array that belongs to the caller. Returning a pointer to a local variable is wrong because the local variable is destroyed when the function returns; the pointer then dangles and using it is undefined behaviour.

A3.3 — Command-line interface. bloodbank find O+ (argc must be 3) and bloodbank issue A- 2 (argc must be 4). Invalid input (no arguments, too many, unknown command, bad blood group, units not a positive number) prints a message on stderr, shows the usage and returns 1. Implemented and tested in Task 5.

Program / Source Code

```
#include <stdio.h>

typedef struct { int id; int age; int days_since; } Donor;
typedef struct { int count; int youngest_id; } EligResult;

int is_eligible(const Donor *d) { return d->age >= 18 && d->age <= 65 && d->days_since >= 90; }

/* Design 1: results returned through pointer parameters */
int eligible_ptr(const Donor d[], int n, int *youngest_id) {
    int count = 0, youngest_age = 1000;
    *youngest_id = -1;
    for (int i = 0; i < n; i++)
        if (is_eligible(&d[i])) {
            count++;
            if (d[i].age < youngest_age) { youngest_age = d[i].age; *youngest_id = d[i].id; }
        }
    return count;
}

/* Design 2: results returned inside a structure */
EligResult eligible_struct(const Donor d[], int n) {
    EligResult r = {0, -1};
    int youngest_age = 1000;
    for (int i = 0; i < n; i++)
        if (is_eligible(&d[i])) {
            r.count++;
            if (d[i].age < youngest_age) { youngest_age = d[i].age; r.youngest_id = d[i].id; }
        }
     return r;
}

/* A3.2: returns a pointer to the element with the lowest stock (NOT to a local variable) */
int *find_lowest(int stock[], int n) {
    int *low = &stock[0];
    for (int i = 1; i < n; i++)
        if (stock[i] < *low) low = &stock[i];
    return low;
}

int main(void) {
    Donor d[5] = {{101, 25, 120}, {102, 17, 200}, {103, 30, 50}, {104, 22, 95}, {105, 45, 300}};
    int yid;
    int c = eligible_ptr(d, 5, &yid);
    printf("Pointer version : eligible = %d, youngest id = %d\n", c, yid);
    EligResult r = eligible_struct(d, 5);
    printf("Struct version : eligible = %d, youngest id = %d\n", r.count, r.youngest_id);

    int stock[8] = {40, 25, 12, 8, 30, 17, 5, 22}; /* O+ O- A+ A- B+ B- AB+ AB- */
    int *p = stock;
    printf("\nstock[8] = {40,25,12,8,30,17,5,22}, p = stock\n");
    printf("p + 3   -> address of stock[%ld]\n", (long)((p + 3) - stock));
    printf("*(p + 3) = %d\n", *(p + 3));
    printf("p[3]     = %d\n", p[3]);
    int v = *p++;   /* use the value, then move p */
    printf("*p++     = %d, p now points to stock[%ld]\n", v, (long)(p - stock));
    int *low = find_lowest(stock, 8);
    printf("\nLowest stock is %d at index %ld\n", *low, (long)(low - stock));
    *low += 50;   /* caller updates the element directly */
    printf("After *low += 50 -> stock[6] = %d\n", stock[6]);
    return 0;
}
```

Output Screenshot

```
Pointer version : eligible = 3, youngest id = 104
Struct version : eligible = 3, youngest id = 104

stock[8] = {40,25,12,8,30,17,5,22}, p = stock
p + 3   -> address of stock[3]
*(p + 3) = 8
p[3]     = 8
*p++     = 40, p now points to stock[1]

Lowest stock is 5 at index 6
After *low += 50 -> stock[6] = 55
```

Analysis / Discussion

The pointer and struct designs give the same answer (3 eligible donors; youngest eligible is 104, age 22; donor 102 is under 18 and 103 donated only 50 days ago). The lowest stock is 5 units at index 6.

Index instead of pointer: if find_lowest returned an index, the caller would write stock[i] += 50. Time O(n) and space O(1) in both versions. The index stays valid if the array is later reallocated; a stored pointer would dangle, so the index version is safer, while the pointer version is more convenient to use.

Result

The function structure, justified prototypes, both multi-value designs and the pointer operations were completed and verified by a running program. I learned when to use const, why pointers are used for output parameters, and why a pointer to a local variable must never be returned.

TASK 2 — Recursion: Digital Root and Power (Parts A2, B1)

Task / Problem Statement

Donor IDs carry a check digit (the digital root of the other digits) and the blood bank computes base^n for a reporting formula. Write recursive designs, trace them, count calls, and find where recursion fails.

Aim

To design, implement and measure recursive functions for the digital root and for power (linear and halving), and to find the limit of recursion.

Concept / Theory

A recursive function solves a problem by calling itself on a smaller input. It needs a base case (stops the recursion) and a recursive case (moves toward the base case). Each call adds a frame to the call stack, so the depth of recursion decides the stack space used.

Divide-and-conquer power uses base^n = (base^(n/2))^2, halving n each time, so only about log n calls are needed. The digital root of n is found by summing digits repeatedly until one digit remains.

Procedure / Algorithm

A2.1 — Recursive pseudocode

```
FUNCTION sum_digits(n)
 IF  n<10THENRETURNn
 RETURN(nMOD10)+sum_digits(nDIV10)

FUNCTION digital_root(n)
 IF  n<10THENRETURNn                          // base case
RETURNdigital_root(sum_digits(n))             // recursive case

FUNCTION power_linear(base, exp)
 IF exp=0THENRETURN1                          // base case
RETURNbase*power_linear(base,exp-1)           // recursive case

FUNCTION power_fast(base, exp)
 IF  exp=0THENRETURN1                         // base case
 half<-power_fast(base,expDIV2)               // ONE recursive call
 IF  expMOD2=0THENRETURNhalf*half
 ELSERETURNhalf*half*base
```

A2.2 — Trace and recurrences

- digital_root(38): sum_digits(38) = 8 + sum_digits(3) = 11, so digital_root(11): sum_digits(11) = 1 + 1 = 2, so digital_root(2) returns 2.
- Deepest call stack for power_fast(2,10): power_fast(2,10), (2,5), (2,2), (2,1), (2,0) = 5 frames. For power_linear(2,4): exp = 4,3,2,1,0 = 5 frames.
- Linear: T(n) = T(n-1) + 1, T(0) = 1, so calls = n + 1. Halving: T(n) = T(n/2) + 1, T(0) = 1, so calls = floor(log n)$_2$ + 2. Two-call version: T(n) = 2T(n/2) + 1, which grows as O(n).

A2.3 — Risks. Without a base case the function calls itself forever; every call uses stack space, so the stack overflows and the program crashes. Use a loop when the recursion depth grows with the input (linear power, n frames) or when the problem is a simple repetition. Recursion is fine when depth is small (log n).

Alternative. Digital root by loop: repeat n = sum_digits(n) until n < 10, time proportional to the number of digits, stack O(1). By formula 1 + (n - 1) % 9 (with 0 handled separately): time O(1), stack O(1). The recursive version uses stack proportional to the digit count.

B1.1 / B1.2 The code below was compared with the Part A pseudocode: the logic is unchanged. Changes: the result type is long long to hold large powers; a global counter calls was added to each power function; digital_root(0) is covered by the base case n < 10; the loop and formula versions were added for comparison.

Tests and what each verifies: n = 0 and 1 (base-case boundaries); 38, 12345, 99999 (ordinary values, one or more passes); 987654321 (needs two passes: 45, then 9); power exp = 0 (base case returns 1), exp = 1, 2^10, 5^3 (ordinary), 2^30 (large result, matches expected 1073741824).

B1.4 — Find the limit. power_linear(1, n) was run for larger and larger n (compiled with -O0, stack limit 8192 KB) until it crashed.

Program / Source Code

```
#include <stdio.h>

long long calls = 0;

int sum_digits(int n) { if (n < 10) return n; return n % 10 + sum_digits(n / 10); }
int digital_root(int n) { if (n < 10) return n; return digital_root(sum_digits(n)); }
int digital_root_loop(int n) { while (n >= 10) n = sum_digits(n); return n; }
int digital_root_formula(int n) { return n == 0 ? 0 : 1 + (n - 1) % 9; }

long long power_linear(long long base, int exp) {
    calls++;
    if (exp == 0) return 1;
    return base * power_linear(base, exp - 1);
}
long long power_fast(long long base, int exp) {
    calls++;
    if (exp == 0) return 1;
    long long half = power_fast(base, exp / 2);
    if (exp % 2 == 0) return half * half;
    return half * half * base;
}
long long power_fast_two(long long base, int exp) {   /* half*half replaced by two calls */
    calls++;
    if (exp == 0) return 1;
    long long r = power_fast_two(base, exp / 2) * power_fast_two(base, exp / 2);
    return (exp % 2) ? r * base : r;
}
long long power_iter(long long base, int exp) {
    long long r = 1;
    while (exp > 0) { if (exp & 1) r *= base; base *= base; exp >>= 1; }
    return r;
}

int main(void) {
    printf("--- B1.1 digital root tests (recursive | loop | formula) ---\n");
    int t[] = {0, 1, 9, 38, 12345, 99999, 987654321};
    for (int i = 0; i < 7; i++)
        printf("n=%-10d %d | %d | %d\n", t[i], digital_root(t[i]), digital_root_loop(t[i]),
               digital_root_formula(t[i]));

    printf("\n--- power tests (linear | fast | iterative) ---\n");
    long long tb[] = {2, 2, 2, 5, 7, 2}; int te[] = {0, 1, 10, 3, 0, 30};
    for (int i = 0; i < 6; i++)
        printf("%lld^%-2d = %lld | %lld | %lld\n", tb[i], te[i],
               power_linear(tb[i], te[i]), power_fast(tb[i], te[i]), power_iter(tb[i], te[i]));

    calls = 0; long long a = power_linear(2, 30); printf("\nlinear: %lld in %lld calls\n", a, calls);
    calls = 0; long long b = power_fast(2, 30);   printf("fast  : %lld in %lld calls\n", b, calls);
    calls = 0; long long c = power_fast_two(2, 30); printf("two-call fast: %lld in %lld calls\n", c, calls);

    printf("\n--- B1.3 call counts (base = 1 so no overflow) ---\n");
    printf("%-8s %-10s %-10s %-12s\n", "n", "linear", "halving", "two-call");
    int ns[] = {10, 100, 1000, 10000};
    for (int i = 0; i < 4; i++) {
        calls = 0; power_linear(1, ns[i]);    long long l = calls;
        calls = 0; power_fast(1, ns[i]);      long long f = calls;
         calls = 0; power_fast_two(1, ns[i]); long long w = calls;
         printf("%-8d %-10lld %-10lld %-12lld\n", ns[i], l, f, w);
    }
    return 0;
}
```

stack_limit.c (used to find the crash point)

```
#include <stdio.h>
#include <stdlib.h>
long long power_linear(long long base, int exp) { if (exp == 0) return 1; return base * power_linear(base, exp -
       1); }
long long power_iter(long long base, int exp) { long long r = 1; while (exp-- > 0) r *= base; return r; }
int main(int argc, char **argv) {
    int n = argc > 2 ? atoi(argv[2]) : 0;
    if (argv[1][0] == 'r') printf("recursive 1^%d = %lld\n", n, power_linear(1, n));
    else                   printf("iterative 1^%d = %lld\n", n, power_iter(1, n));
    return 0;
}
```

Output Screenshot

```
--- B1.1 digital root tests (recursive | loop | formula) ---
n=0           0 | 0 | 0
n=1           1 | 1 | 1
n=9 n=38      9 | 9 | 9
n=12345       2 | 2 | 2
n=99999       6 | 6 | 6
              9 | 9 | 9
  n=987654321 9 | 9 | 9

--- power tests (linear | fast | iterative) ---
2^0  = 1 | 1 | 1
2^1  = 2 | 2 | 2
2^10 = 1024 | 1024 | 1024
5^3  = 125 | 125 | 125
7^0  = 1 | 1 | 1
2^30 = 1073741824 | 1073741824 | 1073741824

linear: 1073741824 in 31 calls
fast : 1073741824 in 6 calls
two-call fast: 1073741824 in 63 calls

--- B1.3 call counts (base = 1 so no overflow) ---
n        linear     halving    two-call
10       11         5          31
100      101        8          255
1000     1001       11         2047
10000    10001      15         32767
```

```
$ ./stack_limit r 261742      ->  recursive 1^261742 = 1
$ ./stack_limit r 262505      ->  Segmentation fault   (exit code 139)
$ ./stack_limit i 1000000000  ->  iterative 1^1000000000 = 1
```

Analysis / Discussion

n	linear calls	n + 1	halving calls	floor(log2 n) + 2	two-call version
10	11	11	5	5	31
100	101	101	8	8	255
1,000	1,001	1,001	11	11	2,047
10,000	10,001	10,001	15	15	32,767

The measured counts match the recurrences exactly. Linear recursion makes n + 1 calls; halving makes only about $\log_2 n$ calls (6 calls for 2^30 against 31).

Changing half * half to two calls gives 63 calls for n = 30 instead of 6, and 32,767 calls for n = 10,000. The same sub-problem is solved twice at every level, so T(n) = 2T(n/2) + 1 = O(n) and the speed advantage of halving is lost.

Recursion limit: the linear version crashed between n = 261,742 and 262,505 on this machine. With an 8 MB stack and about 32 bytes per frame, 8,388,608 / 32 = 262,144 frames, which agrees. The crash is a stack overflow (segmentation fault). The iterative version handled n = 1,000,000,000 using constant stack. The limit depends on the machine and stack size, so rerun stack_limit on your own computer.

Iterative square-and-multiply: time O(log n) like the halving recursion, but stack O(1) instead of O(log n); the linear recursion is O(n) time and O(n) stack.

Result

Both recursive functions worked and matched the iterative and formula versions on all tests. The call counts (31 vs 6 for 2^30) matched the recurrences, and the linear version overflowed the stack near n = 262,000. I learned that a base case and a small recursion depth are essential.

TASK 3 — Dynamic Memory: Growable Donor List (Parts A4, B4)

Task / Problem Statement

The number of donors is not known in advance (50 in a small hospital, 50,000 in the district). Design a donor list that starts small and doubles with realloc, explain memory errors, and decide how a nightly report for 25 hospitals (100 times more donors) should handle memory.

Aim

To implement a growable array with realloc, to prove it has no memory errors, and to choose a memory strategy from measurements.

Concept / Theory

malloc, realloc and free manage heap memory. realloc may move a block, so the old pointer must not be used afterwards and its result must be checked before replacing the old pointer. Doubling the capacity gives amortised O(1) appends because the total copying is O(n).

Memory errors: leak (never freed), dangling pointer (used after free or after the block moved), double free (freed twice). Tools such as AddressSanitizer and LeakSanitizer report them.

Procedure / Algorithm

A4.1 — Pseudocode for adding a donor (capacity doubles)

```
FUNCTION add_donor(list, id, group)
 IF list.size = list.capacity THEN
      IF list.capacity = 0 THEN new_cap <- 2 ELSE new_cap <- list.capacity * 2
      p <- realloc(list.data, new_cap * sizeof(Donor))     // result goes to a TEMPORARY pointer
      IF p = NULL THEN RETURN 0                            // old block still valid, nothing lost
      list.data <- p ; list.capacity <- new_cap
 list.data[list.size] <- (id, group)
list.size <- list.size + 1
 RETURN 1
```

If realloc() fails it returns NULL and leaves the old block untouched, so the function returns 0, the existing donors are still safe and the caller can save them and report the error. If the result were assigned directly to list->data, a failure would overwrite the only pointer with NULL: all donors would be lost and the old block leaked (this is why the temporary pointer p is used).

A4.2 — Memory errors

- **Memory leak:** memory is allocated and never freed, e.g. loading donors in find and forgetting free(list.data) (Leak demo below).
- **Dangling pointer:** using a pointer after its block was freed or moved, e.g. keeping Donor *d = &list.data[0] and then calling add_donor, which may realloc and move the array; d now points to freed memory.
- **Double free:** calling free() twice on the same block, e.g. free(list.data) in main and again inside a cleanup function.
- **Prevention:** realloc through a temporary pointer; one owner and one free_list() that sets data = NULL afterwards; use indexes instead of stored pointers into the array; run with AddressSanitizer / LeakSanitizer.

B4.1 to B4.3 The report program loads donors from a file into the growable list (the doubling code above) and counts eligible donors. A second mode reads the file in batches of 10,000 donors. Both were run for 50 donors (small hospital), 50,000 donors (district now) and 5,000,000 donors (100 times more). The Donor record is 16 bytes (id, group, age, days since donation).

Program / Source Code

donorlist.c (add_donor with realloc)

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;

/* Appends a donor, doubling the capacity when the list is full.
   Returns 1 on success, 0 if memory could not be allocated. */
int add_donor(DonorList *list, int id, const char *group)
{
    if (list->size == list->capacity) {
        int new_cap = (list->capacity == 0) ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, new_cap * sizeof(Donor));   /* keep result in temp pointer */
        if (p == NULL) return 0;          /* old block is still valid in list->data */
        list->data = p;
        list->capacity = new_cap;
    }
    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->size++;
    return 1;
}

int main(void)
{
    DonorList list = {NULL, 0, 0};
    const char *groups[] = {"O+", "A+", "B-", "AB+", "O-"};
    for (int i = 0; i < 5; i++) {
        add_donor(&list, 7001 + i, groups[i]);
        printf("added %d (%s): size = %d, capacity = %d\n",
               list.data[i].id, list.data[i].group, list.size, list.capacity);
    }
    free(list.data);
    return 0;
}
```

report.c (growable list vs batch report, with timing and memory)

```
#include        <stdio.h>
#include       <stdlib.h>
#include       <string.h>
#include         <time.h>
#include <sys/resource.h>

typedef struct { int id; char group[4]; int age; int days; } Donor;   /* 16 bytes */
typedef struct { Donor *data; int size, capacity; } DonorList;

int add_donor(DonorList *l, const Donor *d) {
    if (l->size == l->capacity) {
        int nc = l->capacity == 0 ? 2 : l->capacity * 2;
        Donor *p = realloc(l->data, (size_t)nc * sizeof(Donor));
        if (p == NULL) return 0;
        l->data = p; l->capacity = nc;
    }
    l->data[l->size++] = *d;
    return 1;
}
int eligible(const Donor *d) { return d->age >= 18 && d->age <= 65 && d->days >= 90; }
static double now(void) { return (double)clock() / CLOCKS_PER_SEC * 1000.0; }
static long maxrss_kb(void) { struct rusage u; getrusage(RUSAGE_SELF, &u); return u.ru_maxrss; }

int generate(const char *f, int n) {
    static const char *G[8] = {"O+", "O-", "A+", "A-", "B+", "B-", "AB+", "AB-"};
    FILE *fp = fopen(f, "w"); if (!fp) { perror(f); return 0; }
    srand(42);
    for (int i = 0; i < n; i++) fprintf(fp, "%d %s %d %d\n", 100000 + i, G[rand() % 8], 10 + rand() % 70, rand()
           % 400);
    fclose(fp); return 1;
}

int main(int argc, char **argv) {
    if (argc != 4) { fprintf(stderr, "usage: report gen|mem|batch <file> <n>\n"); return 1; }
    const char *mode = argv[1], *file = argv[2]; int n = atoi(argv[3]);
    if (strcmp(mode, "gen") == 0) return generate(file, n) ? 0 : 1;

    FILE *fp = fopen(file, "r"); if (!fp) { perror(file); return 1; }
    Donor d; int cnt = 0; long total = 0; double t0 = now();
    if (strcmp(mode, "mem") == 0) {                     /* keep ALL donors in a growable list */
        DonorList list = {NULL, 0, 0};
        while (fscanf(fp, "%d %3s %d %d", &d.id, d.group, &d.age, &d.days) == 4)
            if (!add_donor(&list, &d)) { fprintf(stderr, "out of memory\n"); free(list.data); fclose(fp); return
                  1; }
        for (int i = 0; i < list.size; i++) if (eligible(&list.data[i])) cnt++;
        total = list.size;
        printf("mem : donors=%ld eligible=%d list memory=%.2f MB peak RSS=%.1f MB time=%.1f ms\n",
               total, cnt, list.capacity * sizeof(Donor) / 1048576.0, maxrss_kb() / 1024.0, now() - t0);
        free(list.data);
    } else {                                            /* read in batches of 10,000 */
        Donor *batch = malloc(10000 * sizeof(Donor)); int m;
        do {
            m = 0;
            while (m < 10000 && fscanf(fp, "%d %3s %d %d", &batch[m].id, batch[m].group, &batch[m].age,
                  &batch[m].days) == 4) m++;
            for (int i = 0; i < m; i++) if (eligible(&batch[i])) cnt++;
            total += m;
        } while (m == 10000);
        printf("batch: donors=%ld eligible=%d list memory=%.2f MB peak RSS=%.1f MB time=%.1f ms\n",
               total, cnt, 10000 * sizeof(Donor) / 1048576.0, maxrss_kb() / 1024.0, now() - t0);
        free(batch);
    }
    fclose(fp);
    return 0;
}
```

Output Screenshot

```
$ gcc -fsanitize=address,undefined -g donorlist.c -o donorlist && ./donorlist
added 7001 (O+): size = 1, capacity = 2
added 7002 (A+): size = 2, capacity = 2
added 7003 (B-): size = 3, capacity = 4
added 7004 (AB+): size = 4, capacity = 4
added 7005 (O-): size = 5, capacity = 8
(no sanitizer message: no leak, no invalid access)
```

```
$ ./report mem|batch <file> <n>
N = 50
mem  : donors=50 eligible=28 list memory=0.00 MB peak RSS=1.5 MB time=0.0 ms
batch: donors=50 eligible=28 list memory=0.15 MB peak RSS=1.5 MB time=0.0 ms
N = 50000
mem  : donors=50000 eligible=26652 list memory=1.00 MB peak RSS=2.4 MB time=6.7 ms
batch: donors=50000 eligible=26652 list memory=0.15 MB peak RSS=1.7 MB time=6.1 ms
N = 5000000
mem  : donors=5000000 eligible=2657931 list memory=128.00 MB peak RSS=77.9 MB time=670.5 ms
batch: donors=5000000 eligible=2657931 list memory=0.15 MB peak RSS=1.7 MB time=627.4 ms
```

```
$ ./report_asan mem d50000.txt 50000      (AddressSanitizer + LeakSanitizer)
mem  : donors=50000 eligible=26652 list memory=1.00 MB peak RSS=7.8 MB time=12.6 ms
(no error report: all memory freed)
```

```
Leak demo (malloc without free), run with -fsanitize=address:

=================================================================
==301==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 32 byte(s) in 1 object(s) allocated from:
    #0 0x7fd80b6fd9c7 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x55aa4b25921e in main /home/claude/work/leak_demo.c:4
    #2 0x7fd80b22a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: a4a7992a8e66555c8141ab2a08a8465ff6e
    #3 0x7fd80b22a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: a4a7992a8e66555
```

Analysis / Discussion

Donors	Mode	Eligible	List memory	Peak RSS	Time
50	all in memory	28	0.00 MB	1.5 MB	0.0 ms
50	batch	28	0.15 MB	1.5 MB	0.0 ms
50,000	all in memory	26,652	1.00 MB	2.4 MB	6.7 ms
50,000	batch	26,652	0.15 MB	1.7 MB	6.1 ms
5,000,000	all in memory	2,657,931	128 MB capacity	77.9 MB	670.5 ms
5,000,000	batch	2,657,931	0.15 MB	1.7 MB	627.4 ms

The capacity sequence 2, 2, 4, 4, 8 matches the expected output. For 5,000,000 donors the list capacity grows to 8,388,608 x 16 bytes = 128 MB (peak resident memory 77.9 MB because unused pages are not touched). Both modes find the same number of eligible donors.

Why a temporary pointer p: if realloc fails and its result is stored directly in list->data, the old pointer is lost (memory leak and lost data). With p, the old block stays usable on failure.

Fixed +10 growth instead of doubling: for n donors, doubling copies at most about 2n elements in total, O(n) (for 50,000 donors: 2 + 4 + ... + 32,768 = 65,534 copies), but can leave up to about n/2 to n slots unused (65,536 - 50,000 = 15,536 here). Growing by 10 copies 10 + 20 + ... = about $n^2/20$ elements, $O(n^2)$ (about 125 million copies for 50,000 donors), but wastes at most 9 slots.

Production decision (B4.3). For a nightly report (one pass over the data) the batch method is the better choice: it took almost the same time as loading everything (627 ms vs 671 ms for 5 million donors) but needed 0.15 MB instead of up to 128 MB. For interactive searches (find by group, update) the donors of one hospital (about 1 MB for 50,000 donors) can simply be kept in the growable list. If a single donor must be looked up by ID without loading everything, a separate sorted index file with fseek and binary search needs O(1) memory and O(log n) file reads (about 23 reads for 5 million donors) per lookup, compared with O(1) lookup but O(n) memory after loading.

Result

The growable donor list works (capacity 2, 2, 4, 4, 8), handles allocation failure safely and runs clean under AddressSanitizer and LeakSanitizer. Measurements showed batch reading is as fast as loading everything and uses almost no memory, so it is chosen for the nightly report.

TASK 4 — File Storage: Text vs Binary (Parts A5, B3)

Task / Problem Statement

Design the donor file as text and the stock file as binary, update one stock record in place, and compare the two formats with measurements.

Aim

To compare text and binary file storage for correctness, size, speed and update cost, and to update a binary record in place using fseek, fread and fwrite.

Concept / Theory

A text file stores characters and is written with fprintf and read with fscanf or fgets; lines may have different lengths. A binary file stores the raw bytes of a structure and is written with fwrite and read with fread; fixed-size records give direct access with fseek.

Switching between reading and writing on the same file needs an fseek in between. A binary file written on one machine may not be readable on another because of byte order and padding.

Procedure / Algorithm

A5.1 — Choice of file types

- **Donor file = text file**, one donor per line written with fprintf. Names have different lengths, the file can be read and checked by a person, and text is portable between computers.
- **Stock file = binary file of fixed-size records** written with fwrite. Every record has the same size, so record k starts at k x sizeof(record) and fseek can jump straight to it. No text conversion is needed, so reading and writing are fast.

A5.2 — Update in place (pseudocode)

```
FUNCTION update_units(fp, index, units)
 IF fseek(fp, index * sizeof(Stock), SEEK_SET) fails THEN RETURN 0

 IF fread(s, 1 record) does not return 1 THEN RETURN 0       // record does not exist
 s.units <- units
 IF fseek(fp, -sizeof(Stock), SEEK_CUR) fails THEN RETURN 0 // back one record
IF fwrite(s, 1 record) does not return 1 THEN RETURN 0
 RETURN 1
```

Why this is difficult for the text file: lines have different lengths and the position of line k is not known without reading all earlier lines. Changing 9 to 10 makes the line longer and pushes the rest of the file forward, so the whole file has to be rewritten (or copied to a temporary file and renamed).

B3.1 to B3.3 The files.c program writes and reads back 10 records in both formats and compares every field; measures size and read/write time for 1,000, 10,000 and 100,000 donors; and updates the 50,000th record in the binary file (fseek) and in the text file (rewrite). A donor record is {id, name[24], group[4], age} = 36 bytes in binary.

Program / Source Code

stock_update.c (TODO completed)

```
#include <stdio.h>

typedef struct { int group_code; int units; } Stock;   /* fixed-size binary record */

/* Overwrites the units of record number index directly in the binary file. */
int update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fseek(fp, (long)index * sizeof(Stock), SEEK_SET) != 0) return 0;   /* go to record */
    if (fread(&s, sizeof s, 1, fp) != 1) return 0;                         /* read it      */
    s.units = units;                                                       /* change units */
    if (fseek(fp, -(long)sizeof(Stock), SEEK_CUR) != 0) return 0;          /* back 1 record */
    if (fwrite(&s, sizeof s, 1, fp) != 1) return 0;                        /* write again  */
    return 1;
}

int main(void)
{
    FILE *fp = tmpfile();
    if (fp == NULL) { perror("tmpfile"); return 1; }
    Stock initial[5] = {{1, 40}, {2, 25}, {3, 12}, {4, 8}, {5, 30}};
    fwrite(initial, sizeof(Stock), 5, fp);

    printf("update record 3: %s\n", update_units(fp, 3, 20) ? "done" : "failed");
    printf("update record 9: %s\n", update_units(fp, 9, 20) ? "done" : "failed");
    rewind(fp);
    Stock s;
    while (fread(&s, sizeof s, 1, fp) == 1) printf("%d:%d ", s.group_code, s.units);
    printf("\n");
    fclose(fp);
    return 0;
}
```

files.c (text vs binary comparison)

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

typedef struct { int id; char name[24]; char group[4]; int age; } Donor;

static double now(void) { return (double)clock() / CLOCKS_PER_SEC * 1000.0; }   /* milliseconds */

void make(Donor *d, int i) {
    static const char *G[8] = {"O+", "O-", "A+", "A-", "B+", "B-", "AB+", "AB-"};
    d->id = 10000 + i;
    snprintf(d->name, sizeof d->name, "Donor%d", i);
    strcpy(d->group, G[i % 8]);
    d->age = 18 + i % 48;
}
int same(const Donor *a, const Donor *b) {
    return a->id == b->id && strcmp(a->name, b->name) == 0 && strcmp(a->group, b->group) == 0 && a->age ==
          b->age;
}
int write_text(const char *f, Donor *d, int n) {
    FILE *fp = fopen(f, "w"); if (!fp) { perror(f); return 0; }
    for (int i = 0; i < n; i++) fprintf(fp, "%d %s %s %d\n", d[i].id, d[i].name, d[i].group, d[i].age);
    fclose(fp); return 1;
}
int read_text(const char *f, Donor *d, int n) {
    FILE *fp = fopen(f, "r"); if (!fp) { perror(f); return 0; }
    int i = 0;
    while (i < n && fscanf(fp, "%d %23s %3s %d", &d[i].id, d[i].name, d[i].group, &d[i].age) == 4) i++;
    fclose(fp); return i;
}
int write_bin(const char *f, Donor *d, int n) {
    FILE *fp = fopen(f, "wb"); if (!fp) { perror(f); return 0; }
    fwrite(d, sizeof(Donor), n, fp); fclose(fp); return 1;
}
int read_bin(const char *f, Donor *d, int n) {
    FILE *fp = fopen(f, "rb"); if (!fp) { perror(f); return 0; }
    int r = (int)fread(d, sizeof(Donor), n, fp); fclose(fp); return r;
}
long fsize(const char *f) { FILE *fp = fopen(f, "rb"); fseek(fp, 0, SEEK_END); long s = ftell(fp); fclose(fp);
      return s; }

/* update record k (0-based) in place in the binary file */
int update_bin(const char *f, int k, int new_age) {
    FILE *fp = fopen(f, "r+b"); if (!fp) return 0;
    Donor d;
    fseek(fp, (long)k * sizeof d, SEEK_SET);
    if (fread(&d, sizeof d, 1, fp) != 1) { fclose(fp); return 0; }
    d.age = new_age;
    fseek(fp, -(long)sizeof d, SEEK_CUR);
    fwrite(&d, sizeof d, 1, fp);
    fclose(fp); return 1;
}
/* update record k in the text file by rewriting the whole file */
int update_text(const char *f, int k, int new_age, int n) {
    Donor *d = malloc(sizeof(Donor) * n); if (!d) return 0;
    int got = read_text(f, d, n);
    if (k >= got) { free(d); return 0; }
    d[k].age = new_age;
    int ok = write_text(f, d, got);
    free(d); return ok;
}

int main(void) {
    /* B3.1 correctness on 10 records */
    Donor a[10], b[10], c[10];
    for (int i = 0; i < 10; i++) make(&a[i], i);
    write_text("t10.txt", a, 10); write_bin("t10.bin", a, 10);
    int rt = read_text("t10.txt", b, 10), rb = read_bin("t10.bin", c, 10), okt = (rt == 10), okb = (rb == 10);
    for (int i = 0; i < 10; i++) { okt = okt && same(&a[i], &b[i]); okb = okb && same(&a[i], &c[i]); }
    printf("B3.1 text file : %d records read back, identical = %s\n", rt, okt ? "YES" : "NO");
    printf("      binary file: %d records read back, identical = %s\n\n", rb, okb ? "YES" : "NO");

    /* B3.2 size and time */
    printf("B3.2 %-8s | %-9s %-10s %-10s | %-9s %-10s %-10s\n", "records", "text size", "write ms", "read ms",
          "bin size", "write ms", "read ms");
    int sizes[3] = {1000, 10000, 100000};
    for (int s = 0; s < 3; s++) {
        int n = sizes[s];
        Donor *x = malloc(sizeof(Donor) * n), *y = malloc(sizeof(Donor) * n);
        for (int i = 0; i < n; i++) make(&x[i], i);
        double t0 = now(); write_text("big.txt", x, n); double t1 = now(); read_text("big.txt", y, n); double t2
              = now();
        write_bin("big.bin", x, n); double t3 = now(); read_bin("big.bin", y, n); double t4 = now();
```

```
        printf("      %-8d | %-9ld %-10.2f %-10.2f | %-9ld %-10.2f %-10.2f\n", n, fsize("big.txt"), t1 - t0, t2
              -t1,fsize("big.bin"), t3 - t2, t4 - t3);
        free(x); free(y);
    }

    /*B3.3updatethe50,000th record (index 49999) of the 100,000-record files */
    doublet0=now();update_bin("big.bin", 49999, 99); double t1 = now();
    update_text("big.txt", 49999, 99, 100000); double t2 = now();
    printf("\nB3.3  update 50,000th record: binary fseek = %.3f ms, text rewrite = %.2f ms\n", t1 - t0, t2 -
          t1);
    return 0;
}
```

Output Screenshot

```
$ ./stock_update
update record 3: done
update record 9: failed
1:40 2:25 3:12 4:20 5:30
```

```
$ ./files
B3.1 text file : 10 records read back, identical = YES
      binary file: 10 records read back, identical = YES

B3.2 records | text size write ms     read  ms   | bin size write ms    read ms
      1000     | 21140     0.23       0.13       | 36000     0.12       0.00
      10000    | 221390    0.90       1.19       | 360000    0.10       0.03
      100000   | 2323890   8.98       12.22      | 3600000   0.86       0.71

B3.3 update 50,000th record: binary fseek = 0.013 ms, text rewrite = 21.32  ms
```

Analysis / Discussion

Why updating record 9 fails and the file is not damaged: the file holds only records 0 to 4. fseek beyond the end is allowed, but fread then returns 0, so update_units returns 0 before any fwrite is made. Nothing is written, so the file stays exactly as it was (output 1:40 2:25 3:12 4:20 5:30, only record 3 changed).

Records	Text size	Binary size	Text write / read	Binary write / read
1,000	21,140 B	36,000 B	0.23 / 0.13 ms	0.12 / 0.00 ms
10,000	221,390 B	360,000 B	0.90 / 1.19 ms	0.10 / 0.03 ms
100,000	2,323,890 B	3,600,000 B	8.98 / 12.22 ms	0.86 / 0.71 ms

Observations. Both formats gave identical data on the 10-record test. Here the binary file is bigger (36 bytes per record because name[24] is always stored in full, against about 23 bytes per text line), yet it is roughly 10 to 15 times faster to read and write because there is no number-to-text conversion. Updating the 50,000th record took about 0.01 ms with fseek (O(1)) against about 21 ms for rewriting the text file (O(n)), roughly 1,000 times slower. Times vary slightly from run to run and between computers.

Engineering trade-off. Binary files are not always portable: another computer or compiler may use a different int size, byte order (endianness) or structure padding, so the same bytes can be read wrongly. Text is portable and human-readable but slower and not updatable in place. Recommendation: use a **text (CSV-style) file for donors** (variable length, shared between hospitals, easy to check) and a **fixed-record binary file for stock** (numbers only, small, updated very often).

One fwrite for the whole array: it needs 1 library call instead of n calls and the data must first be held in an array of n x sizeof(record) bytes (for 8 stock records this is only 64 bytes). Time stays O(n) because the same bytes are written, but with less call overhead; for huge files the memory cost of holding the whole array is the limit.

Stock kept in a text file and fully rewritten after each update: time per update O(n) against O(1) for in-place fseek; the text file size varies with the numbers while the binary size is exactly n x sizeof(record).

Result

The in-place update worked (record 3 done, record 9 failed without damage) and both formats read back identical data. Binary was about 1,000 times faster for updating one record, so binary is recommended for stock and text for donors.

TASK 5 — Adversarial Inputs and Error Handling (Part B2)

Task / Problem Statement

Give the program deliberately bad input: a donor name longer than its array (scanf and fgets), a missing or unwritable data file, wrong command-line arguments, and run it under a memory checker.

Aim

To find how the program fails on bad input, to correct the weaknesses, and to show that errors are reported cleanly.

Concept / Theory

Reading input into a fixed array without a limit causes a buffer overflow, a memory error that corrupts the stack. fgets limits the input to the array size. A function such as fopen returns NULL on failure and perror prints the system's reason. The values of argc and argv must be checked before use.

AddressSanitizer (gcc -fsanitize=address) and valgrind find memory errors such as overflows, leaks and use after free while the program runs.

Procedure / Algorithm

1. Write a donor name program with a small array (8 bytes) using scanf("%s") and run it with a 52-character name.
2. Run the same input with a fixed version that uses fgets with the array size.
3. Build the command-line program bloodbank.c with checks on argc, argv, the blood group and the units, and use perror() when a file cannot be opened.
4. Run it with a missing file, no arguments, too many arguments and an invalid blood group.
5. Compile with gcc -fsanitize=address and report the memory errors found and fixed (valgrind was not available here, so the compiler sanitizer allowed by the task was used).

Program / Source Code

name_bad.c (unsafe)

```
#include <stdio.h>
int main(void) {

    char name[8];
    printf("Enter donor name: ");
    scanf("%s", name);                /* no limit: unsafe */
    printf("Name stored: %s\n", name);
    return 0;
}
```

name_good.c (fixed with fgets)

```
#include <stdio.h>
#include <string.h>
int main(void) {

    char name[8];
    printf("Enter donor name: ");
    if (fgets(name, sizeof name, stdin) == NULL) return 1;
    int cut = strchr(name, '\n') == NULL;      /* no newline => input was longer than the array */
    name[strcspn(name, "\n")] = '\0';
    printf("Name stored: %s%s\n", name, cut ? " (input was truncated to fit)" : "");
    return 0;
}
```

bloodbank.c (command-line interface with error handling)

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>

static const char *GROUPS[8] = {"O+", "O-", "A+", "A-", "B+", "B-", "AB+", "AB-"};
static int stock[8] = {40, 25, 12, 8, 30, 17, 5, 22};

int group_index(const char *g) {
    for (int i = 0; i < 8; i++) if (strcmp(g, GROUPS[i]) == 0) return i;
    return -1;
}
void usage(void) {
    fprintf(stderr, "Usage:\n bloodbank find <group>\n bloodbank issue <group> <units>\n");
}
int find_donors(const char *group) {
    FILE *fp = fopen("donors.txt", "r");
    if (fp == NULL) { perror("donors.txt"); return 1; }
    int id, age, days; char name[32], g[8], line[128];
    int found = 0;
    while (fgets(line, sizeof line, fp)) {
        if (sscanf(line, "%d %31s %7s %d %d", &id, name, g, &age, &days) != 5) continue;
        if (strcmp(g, group) == 0 && age >= 18 && age <= 65 && days >= 90) {
            printf("Eligible: %d %s (%s)\n", id, name, g); found++;
        }
    }
    fclose(fp);
    if (found == 0) printf("No eligible donors for %s\n", group);
    return 0;
}
int main(int argc, char *argv[]) {
    if (argc < 3) { fprintf(stderr, "Error: too few arguments\n"); usage(); return 1; }
    int gi = group_index(argv[2]);
    if (gi < 0) { fprintf(stderr, "Error: invalid blood group '%s'\n", argv[2]); usage(); return 1; }
    if (strcmp(argv[1], "find") == 0) {
        if (argc != 3) { fprintf(stderr, "Error: too many arguments\n"); usage(); return 1; }
        return find_donors(argv[2]);
    }
    if (strcmp(argv[1], "issue") == 0) {
        if (argc != 4) { fprintf(stderr, "Error: 'issue' needs <group> <units>\n"); usage(); return 1; }
        char *end; long u = strtol(argv[3], &end, 10);
        if (*end != '\0' || u <= 0) { fprintf(stderr, "Error: units must be a positive number\n"); return 1; }
        if (u > stock[gi]) { fprintf(stderr, "Error: only %d units of %s in stock\n", stock[gi], argv[2]);
              return 1; }
        stock[gi] -= (int)u;
        printf("Issued %ld units of %s. Remaining: %d\n", u, argv[2], stock[gi]);
        return 0;
    }
    fprintf(stderr, "Error: unknown command '%s'\n", argv[1]); usage(); return 1;
}
```

Output Screenshot

```
sample    donors.txt:
7001 Asha O+ 25 120
7002 Ravi A+ 17 200
7003 Meena B- 30 50
7004 Kiran O+ 22 95
7005 Latha AB+ 45 300
```

```
$ ./bloodbank
Error: too few arguments
Usage:
 bloodbank find <group>
bloodbank issue <group> <units>
[exit 1]

$ ./bloodbank find O+ extra
Error: too many arguments
Usage:

$ ./bloodbank find Z+
Error: invalid blood group 'Z+'

$ ./bloodbank issue A- 100
Error: only 8 units of A- in stock

$ ./bloodbank issue A- 3
Issued 3 units of A-. Remaining: 5

$ ./bloodbank find O+
Eligible: 7001 Asha (O+)
Eligible: 7004 Kiran (O+)

$ ./bloodbank find O+   (donors.txt missing)
donors.txt: No such file or directory
[exit 1]
$ ./bloodbank find O+   (ASan clean run)
Eligible: 7001 Asha (O+)
Eligible: 7004 Kiran (O+)
[exit 0]
```

```
$ ./name_bad (plain build) input: 52-letter name
*** stack smashing detected ***: terminated
[exit 134]

$ ./name_bad (gcc -fsanitize=address) same input
=================================================================
==228==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f6518b00028 at pc 0x7f651b091ad0 bp 0x
WRITE of size 53 at 0x7f6518b00028 thread T0
    #0 0x7f651b091acf in scanf_common ../../../../src/libsanitizer/sanitizer_common/sanitizer_common_inter
    ...
Address 0x7f6518b00028 is located in stack of thread T0 at offset 40 in frame

$ ./name_good (fgets) same input
Enter donor name: Name stored: Abcdefg (input was truncated to fit)
[exit 0]
$ ./name_good input: Asha
Enter donor name: Name stored: Asha
```

```
$ ./savetest    (opening a directory for writing, as an unwritable file)
save donors: Is a directory
```

Analysis / Discussion

- **Long name with scanf("%s"):** scanf has no size limit, so the 52 characters were written past the 8-byte array. The plain build ended with "stack smashing detected" (abort); AddressSanitizer reported **stack-buffer-overflow, WRITE of size 53** at name_bad.c line 5. Without protection this can overwrite other variables or the return address.
- **With fgets(name, sizeof name, stdin):** at most 7 characters plus the end marker are stored, the extra input is not written, and the program tells the user the name was truncated. **Correction made:** scanf("%s") replaced by fgets.
- **Missing or unwritable file:** fopen returns NULL and the program prints the reason with perror ("donors.txt: No such file or directory", "save donors: Is a directory") and returns 1 instead of crashing.
- **Wrong arguments:** none, too many, an unknown group (Z+), too many units (only 8 units of A- in stock) all give a clear message on stderr and exit code 1; valid commands work.
- **Sanitizer on bloodbank.c** (built with -fsanitize=address): no memory errors found; the fclose and the fgets size limit prevent leaks and overflows.

Alternative: getline() instead of fgets. getline allocates a buffer of the needed size, so no input is truncated. Time to read a line of length L is O(L) (plus occasional buffer growth) and the heap space is O(L) instead of a fixed array. The caller must free the buffer. A maximum length should still be checked so that a huge line cannot use all memory.

Result

The deliberate overflow was found by AddressSanitizer and corrected with fgets; missing files and wrong arguments are reported with clear messages instead of crashes. I learned to treat every input as untrusted and to test with a memory checker.

Test Cases / Validation Table

Differentinputs,expectedresults,actualresultsand status (actual results are from the program runs shown above).

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	digital_root(38)	2	2	Pass
2	digital_root(0) and (1)	0 and 1	0 and 1	Pass
3	power_fast(2,30)	1073741824	1073741824	Pass
4	calls for power_linear(2,30) / power_fast(2,30)	31 / 6	31 / 6	Pass
5	power_linear(1, 262505) (very deep recursion)	Stack overflow (expected limit about 262,144)	Segmentation fault	Pass
6	Add 5 donors to an empty growable list	capacity 2, 2, 4, 4, 8	capacity 2, 2, 4, 4, 8	Pass
7	update_units record 3 and record 9	done / failed, file intact	done / failed, 1:40 2:25 3:12 4:20 5:30	Pass
8	10 donor records written and read back (text and binary)	Identical data	Identical: YES / YES	Pass
9	bloodbank with no arguments / find Z+	Error message and usage, exit 1	Error message and usage, exit 1	Pass
10	bloodbank find O+ with donors.txt missing	perror message, no crash	donors.txt: No such file or directory	Pass
11	52-letter name with scanf / with fgets	Overflow detected / name truncated safely	stack-buffer-overflow / Abcdefg (truncated)	Pass
12	Report on 5,000,000 donors (memory vs batch)	Same eligible count	2,657,931 in both	Pass

Final Submission Checklist

[x] Studentandcourse detailscompleted

[x] Task statement, aim, theory, and algorithm included

[x] Program code is readable and runs successfully

[x] Output screenshot included (program output captured from a compile-and-run; replace with your own screenshot if your lab requires it)

[x] Analysis and result written (please read and rewrite in your own words)

[x] Test cases completed where applicable

[ ] GitHub repository link added if required (not added: no link provided)